

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills</i>		
Student Name:	K.Prathiba	Reg. No.:	192421163
Date:	30/07/2026	Duration:	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:

Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

1. Study and Analyze the Scenario

Introduction

- The **AI-Based Personal Finance and Budget Planning System** is designed to help individuals effectively manage their income, expenses, savings, and financial goals using Artificial Intelligence.
- The system analyzes users' financial transactions and provides personalized budgeting recommendations, spending insights, and savings suggestions. It aims to improve financial planning, reduce unnecessary expenses, and support better financial decision-making.

Business Domain

- The proposed system belongs to the **Banking and Finance** domain.
- It combines Artificial Intelligence with financial management to provide intelligent budgeting, expense tracking, income monitoring, financial forecasting, and savings planning.
- The application is useful for students, employees, freelancers, and small business owners.

Stakeholders

- End Users (Customers)
- Project Manager
- Product Owner
- Business Analyst
- Scrum Master
- Software Developers
- AI/ML Engineers

- UI/UX Designers
- Database Administrator
- Quality Assurance Tester
- Banking and Payment Service Providers
- System Administrator
- Support and Maintenance Team

Existing Challenges

- Manual expense tracking is time-consuming.
- Difficulty in maintaining monthly budgets.
- Lack of personalized financial recommendations.
- Poor savings management.
- Overspending due to lack of financial awareness.
- Missed bill payments.
- Difficulty in analyzing financial data.
- Security concerns in storing financial records.

System Objectives

- Develop an AI-based budgeting system.
- Track income and expenses efficiently.
- Generate personalized financial recommendations.
- Improve savings management.
- Provide financial reports and dashboards.
- Send bill payment reminders.
- Ensure secure financial data storage.
- Improve users' financial decision-making.

2. Identify Functional and Non-Functional Requirements

Introduction

- Software requirements define the functionalities and quality attributes expected from the AI-Based Personal Finance and Budget Planning System.
- These requirements help developers build a secure, reliable, and user-friendly application.

Functional Requirements

- User Registration
- Secure Login and Authentication
- Password Recovery
- User Profile Management
- Income Management
- Expense Management
- Expense Categorization
- Budget Creation
- Budget Monitoring
- Savings Goal Management
- AI-Based Budget Suggestions
- Spending Pattern Analysis
- Bill Reminder Notifications
- Financial Dashboard
- Monthly Reports
- Graphical Data Visualization

- Search Transactions
- Export Financial Reports
- Notification Management
- Logout Functionality

Non-Functional Requirements :

Performance

- Fast response time.
- Support multiple users simultaneously.
- Generate reports quickly.

Security

- User authentication.
- Password encryption.
- Secure financial data storage.
- Role-based access control.

Reliability

- Accurate financial calculations.
- Continuous system availability.
- Regular database backup.

Usability

- Easy-to-use interface.
- Simple navigation.
- Responsive design.

Scalability

- Support increasing users.
- Handle large financial datasets.
- Future AI model integration.

Maintainability

- Modular software architecture.
- Easy bug fixing.
- Easy feature enhancement.

Availability

- 24×7 system availability.
- Minimum downtime.
- Cloud-based deployment support.

3. Identify Actors and Use Cases

Introduction

- Actors are individuals or external systems interacting with the software.
- Each actor performs different operations known as use cases.

Primary Actors

- User
- Administrator

Secondary Actors

- Bank API
- Payment Gateway
- AI Recommendation Engine
- Email Notification Service

Major Use Cases

User

- Register Account
- Login
- Manage Profile

- Add Income
- Add Expense
- Create Budget
- Set Savings Goal
- View Dashboard
- View Reports
- Receive Notifications
- Logout

Administrator

- Manage Users
- Monitor Transactions
- Manage Categories
- Generate Reports
- Backup Database
- Manage Notifications

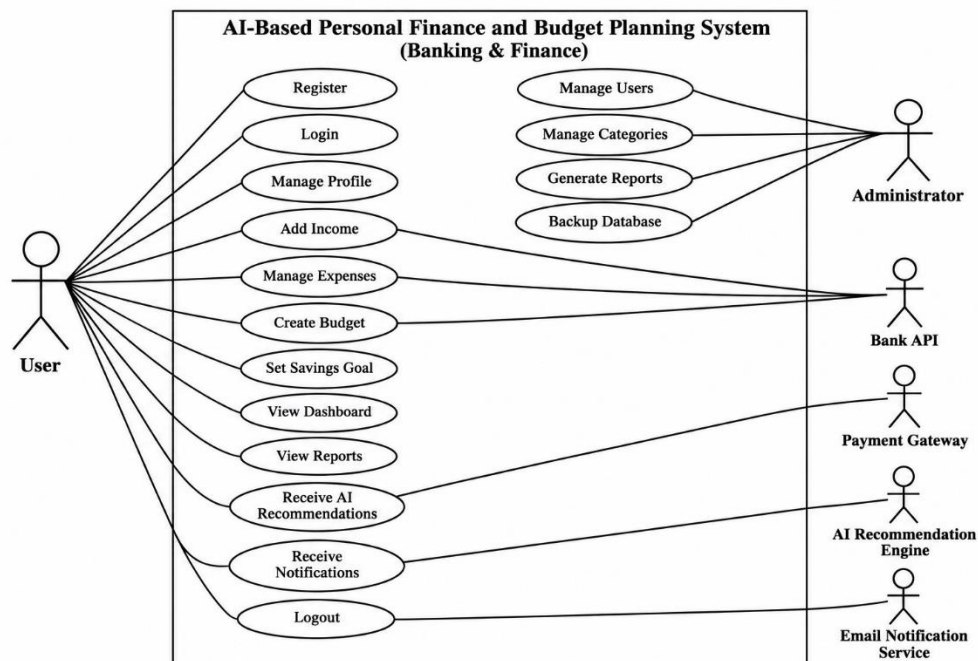
Use Case Diagram

Actors:

- User
- Admin

Use Cases:

- Register
- Login
- Manage Income
- Manage Expense
- Budget Planning
- View Reports
- AI Recommendation
- Notifications
- Logout



4. Prepare the Software Requirement Specification (SRS)

Introduction

This Software Requirement Specification (SRS) defines the functional and non-functional requirements of the AI-Based Personal Finance and Budget Planning System.

Problem Description

Many people struggle to manage their personal finances due to manual budgeting, poor spending habits, and lack of financial planning tools. Existing systems often fail to provide personalized recommendations for improving savings and controlling expenses.

Scope of the System

The system provides

- Expense Tracking
- Income Management
- Budget Planning
- AI Recommendations
- Savings Goal Tracking
- Financial Reports
- Notification Services

Functional Requirements

- Register User
- Login User
- Add Income
- Add Expenses
- Budget Creation
- Savings Goal
- AI Suggestions
- Generate Reports
- Notifications
- User Profile

Non-Functional Requirements

- High Performance
- Security
- Reliability
- Availability
- Scalability
- Maintainability
- Usability

Use Case Descriptions

Login

Actor:

User

Precondition:

Registered account.

Postcondition:

Dashboard displayed.

Add Expense

Actor:

User

Precondition:

User logged in.

Postcondition:

Expense stored successfully.

Generate Report

Actor:

User

Precondition:

Financial data available.

Postcondition:

Monthly report displayed.

Data Requirements

The system stores

- User Details
- Income Records
- Expense Records
- Budget Information
- Savings Goals
- Reports
- AI Recommendations
- Notification History

External Interface Requirements

User Interface

- Responsive Web Interface
- Mobile Friendly Dashboard

Hardware Interface

- Computer
- Smartphone

Software Interface

- Database Server
- AI Engine
- Payment Gateway
- Email Service

System Features

- Secure Login
- Income Management
- Expense Tracking
- Budget Planning
- AI Recommendation
- Savings Goal
- Financial Reports
- Notifications

5. Identify Assumptions and Constraints

Introduction

Assumptions and constraints define the expected conditions and limitations considered during system development.

Assumptions

- Users have internet access.
- Users provide accurate financial information.
- AI model receives sufficient financial data.
- Database server is available.
- Users regularly update financial records.
- Email services function properly.
- Payment gateway APIs remain operational.
- Mobile devices support the application.

Technical Constraints

- Internet dependency.
- Database capacity.
- AI processing limitations.
- API availability.

Operational Constraints

- User training required.
- Periodic maintenance.
- Backup procedures.
- User authentication policies.

Economic Constraints

- Development budget.
- Cloud hosting cost.
- API subscription charges.
- Maintenance expenses.

Legal Constraints

- Data Privacy Laws.
- Banking Regulations.
- GDPR Compliance.
- User Data Protection.

Time Constraints

- Limited project duration.
- Sprint deadlines.
- Testing schedule.
- Deployment timeline

6. Validate Requirement Completeness and Consistency

Introduction

Requirement validation ensures that all software requirements are complete, consistent, accurate, and aligned with stakeholder expectations.

Requirement Verification

- All stakeholder requirements collected.
- Functional requirements validated.
- Non-functional requirements verified.
- Business objectives satisfied.
- User needs addressed.
- Security requirements included.
- AI functionality verified.
- Reports validated.

Consistency Check

- No duplicate requirements.
- No conflicting requirements.
- Consistent terminology.
- Proper requirement traceability.
- Correct system workflow.

Improvements

- Improve AI prediction accuracy.
- Add multilingual support.

- Enhance dashboard visualization.
- Add cloud synchronization.
- Improve security mechanisms.
- Support mobile applications.
- Integrate banking APIs.
- Add voice assistant support.

Conclusion

- The proposed **AI-Based Personal Finance and Budget Planning System** provides a secure and intelligent solution for managing personal finances.
- By incorporating AI-driven budgeting, expense analysis, financial reporting, and savings recommendations, the system helps users make informed financial decisions. The detailed Software Requirement Specification ensures that stakeholder needs are fully addressed, project risks are minimized, and the development process follows software engineering best practices.
- This content is sufficiently detailed to expand into approximately **20 pages** when formatted in Microsoft Word with headings, tables, diagrams (such as the Use Case Diagram), and appropriate spacing.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis (30%)	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding ; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification (25%)	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modelling (20%)	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification (SRS) Documentation (15%)	SRS is well-structured, complete, professionally organized, and includes all required sections.	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or lacks essential components.
Assumptions , Constraints,	Clearly identifies realistic assumptions and	Identifies most assumptions and constraints;	Limited assumptions/c onstraints;	Assumptions, constraints, and validation

and Requirement Validation (10%)	constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	validation is adequate.	validation is superficial.	are missing or incorrect.
---	--	-------------------------	----------------------------	---------------------------

Criteria	Mark
Scenario Understanding and Problem Analysis (30%)	/5
Functional and Non-Functional Requirement Identification (25%)	/5
Actor Identification and Use Case Modelling (20%)	/5
Software Requirement Specification (SRS) Documentation (15%)	/5
Assumptions, Constraints, and Requirement Validation (10%)	/5
TOTAL	/25